A Unified Interactive Platform for Teaching Formal Languages and Automata Theory

Bujor Mihai Alexandru
bujormihaialexandru@gmail.com
Babeș-Bolyai University

Paper Outline

Abstract

Students struggle with formal languages theory due to abstract concepts and weak understanding of relationships between formalisms. We present a unified web-based platform supporting the entire Chomsky hierarchy with interactive construction, step-by-step simulation, and conversion algorithm visualization. A controlled experiment using exam performance evaluates whether comprehensive, visual, and interactive tooling improves learning outcomes compared to existing fragmented approaches.

1. Introduction

- **Problem:** Students memorize definitions but fail to understand connections (regex ↔ DFA ↔ grammar equivalence)
- **Current tools:** JFLAP comprehensive but dated UI and desktop-only; modern web tools limited in scope
- **Solution:** Unified platform combining: comprehensive coverage (DFA → Turing machines), interactive construction, animated execution, conversion visualization
- **Research question:** Does such a platform improve exam performance and conceptual understanding?
- **Contributions:** Empirical evaluation, design principles, open educational resource

2. Background and Related Work

2.1 Pedagogical Challenges

Abstract formalism and weak relational understanding are major barriers [1], [2]. Students struggle with execution tracing [3] and connections between models [4].

2.2 Educational Technology

Active engagement crucial for visualization effectiveness [5], [6], [7]. Interactive construction [8], worked examples [9], and immediate feedback [10] support learning. Multiple representations aid understanding [11] but generic approaches have mixed results [12].

2.3 Existing Tools

JFLAP [13]: Most comprehensive (DFA, NFA, PDA, TM, grammars, conversions) but Java desktop app with complex interface. **Web simulators** [14]: Simple, accessible but limited scope (typically DFA only). **Gap:** No modern tool combining comprehensiveness, usability, and pedagogical design.

2.4 Cognitive Factors

Cognitive load theory [15] suggests well-designed interfaces reduce extraneous load. Analogical reasoning [16] and cognitive technologies [17] enhance learning.

3. Research Questions

RQ1: Does the platform improve exam performance on automata theory topics?

RQ2: Does it improve understanding of relationships between formalisms (e.g., regex ↔ DFA equivalence)?

RQ3: Which features contribute most to learning: conversion visualization, interactive editor, or execution animation?

RQ4: What is the usability and perceived value of the platform?

4. Platform Design

4.1 Coverage

Type 3: DFA, NFA, ε-NFA, regex, regular grammars
Type 2: PDA, CFG, parsers
Type 1/0: LBA, Turing machines
Conversions: Regex ↔ NFA ↔ DFA ↔ Grammar, CFG ↔ PDA (step-by-step)

4.2 Key Features

- **Interactive editor:** Drag-and-drop construction for all models
- **Animated simulation:** Synchronized state/input/stack/tape visualization
- **Conversion visualization:** Algorithm steps with correspondence highlighting
- **Type detection:** Automatic classification and validation
- **Example library:** Progressive complexity with real-world patterns
- **Immediate feedback:** Error explanations and hints

4.3 Design Goals

Comprehensive (entire Chomsky hierarchy), accessible (web-based), visual (rich animations), interactive (construction-first), pedagogical (explicit relationships).

5. Evaluation

5.1 Study Design

Type: Controlled experiment with exam as primary outcome

Participants: Students in formal languages course (hopefully N=80-100)

Groups: Random assignment to control (existing tools) vs treatment (unified platform)

Duration: 3-5 weeks before exam period

5.2 Procedure

Pre-Intervention:

- Background survey (programming experience, prior knowledge)
- Random assignment stratified by experience

Intervention (3-5 weeks):

- Both groups: identical lecture content
- Control: existing tools (JFLAP, online regex testers, paper exercises)
- Treatment: unified platform exclusively
- Weekly: cognitive load surveys [18], usage analytics

Assessment:

- **Primary outcome:** Course exam performance (questions on execution, design, conversions, theory)
- **Secondary outcomes:**
  - Relational understanding tasks: "Explain regex ↔ DFA equivalence"
  - Conversion task: NFA → minimal DFA (timed)

- Usability: SUS [19] (treatment group)
- Interviews: N=15 per group on learning experience

5.3 Metrics

- Exam scores (overall and by question type: execution/design/conversions/theory)
- Relational understanding rubric (0-4 scale: none → deep explanation)
- Conversion accuracy and time
- Cognitive load (NASA-TLX)
- SUS score and feature usage patterns

5.4 Analysis

- Primary: t-test comparing exam scores (control vs treatment)
- ANCOVA controlling for prior experience
- Subgroup analysis by question type
- Regression: feature usage → performance (treatment group)
- Qualitative: thematic analysis of interviews

6. Expected Results

- 15-25% higher exam scores (treatment group)
- Particularly strong on conversion and relational understanding questions
- High usability (SUS > 75)
- Qualitative: students report better grasp of connections
- Feature analysis: conversion visualization most correlated with performance

7. Discussion

Well-designed comprehensive tools can improve learning by making abstract concepts concrete and relationships explicit. Success factors: visual execution, interactive construction, conversion algorithm transparency, unified interface.

Limitations: Single institution, short duration, confounding factors (motivation, time-on-task).

Implications: Informs tool design for other abstract CS topics (compilers, verification).

8. Contributions

C1: Comprehensive platform (DFA → TM) with modern web-based design

C2: Empirical evidence using real exam performance (ecological validity)

C3: Design principles: comprehensiveness + usability + visualization + interactivity

C4: Open educational resource

Bibliography

Bibliography

[1] S. H. Rodger, "Increasing Engagement in Automata Theory with JFLAP," in *Proceedings of the 37th SIGCSE Technical Symposium on Computer Science Education*, 2006, pp. 403–407. doi: 10.1145/1121341.1121479.

[2] D. X. Du and M. N. S. Swamy, "Teaching Formal Languages and Automata Using JFLAP," *Journal of Computing Sciences in Colleges*, vol. 25, no. 4, pp. 111–119, 2010.

[3] R. Lister *et al.*, "A Multi-National Study of Reading and Tracing Skills in Novice Programmers," *ACM SIGCSE Bulletin*, vol. 36, no. 4, pp. 119–150, 2004, doi: 10.1145/1041624.1041673.

[4] J. D. Novak and A. J. Cañas, "The Theory Underlying Concept Maps and How to Construct Them," *Florida Institute for Human and Machine Cognition*, vol. 1, pp. 1–31, 2006.

[5] T. L. Naps *et al.*, "Exploring the Role of Visualization and Engagement in Computer Science Education," *ACM SIGCSE Bulletin*, vol. 34, no. 2, pp. 131–152, 2002, doi: 10.1145/543812.543829.

[6] C. D. Hundhausen, S. A. Douglas, and J. T. Stasko, "A Meta-Study of Algorithm Visualization Effectiveness," *Journal of Visual Languages and Computing*, vol. 13, no. 3, pp. 259–290, 2002, doi: 10.1006/jvlc.2002.0237.

[7] V. Karavirta, A. Korhonen, L. Malmi, and J. Stasko, "State of the Art in Algorithm Visualization," in *Proceedings of the 2006 ACM SIGCSE Technical Symposium on Computer Science Education*, 2006, pp. 530–534. doi: 10.1145/1121341.1121513.

[8] S. Papert, *Mindstorms: Children, Computers, and Powerful Ideas*. New York, NY: Basic Books, 1980.

[9] J. Sweller and G. A. Cooper, "The Use of Worked Examples as a Substitute for Problem Solving in Learning Algebra," *Cognition and Instruction*, vol. 2, no. 1, pp. 59–89, 1992, doi: 10.1207/s1532690xci0201_3.

[10] S. Narciss, "Feedback Strategies for Interactive Learning Tasks," *Handbook of Research on Educational Communications and Technology*, vol. 3, pp. 125–144, 2008.

[11] S. Ainsworth, "DeFT: A Conceptual Framework for Considering Learning with Multiple Representations," *Learning and Instruction*, vol. 16, no. 3, pp. 183–198, 2006, doi: 10.1016/j.learninstruc.2006.03.001.

[12] J. Sorva, V. Karavirta, and L. Malmi, "A Review of Generic Program Visualization Systems for Introductory Programming Education," *ACM Transactions on Computing Education*, vol. 13, no. 4, pp. 1–64, 2013, doi: 10.1145/2490822.

[13] S. H. Rodger and T. W. Finley, "JFLAP: An Interactive Formal Languages and Automata Package," *Jones and Bartlett Publishers*, 2006.

[14] I. Borajek, "Automaton Simulator: A Web-Based Tool for Teaching Formal Languages," in *Proceedings of the 2015 Central European Conference on Information and Intelligent Systems*, 2015, pp. 129–135.

[15] J. Sweller, "Cognitive Load During Problem Solving: Effects on Learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988, doi: 10.1207/s15516709cog1202_4.

[16] A. N. Kumar, "Analogical Reasoning in Learning Programming," in *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*, 2015, pp. 53–58. doi: 10.1145/2676723.2677299.

[17] R. D. Pea, "Cognitive Technologies for Mathematics Education," in *Cognitive Science and Mathematics Education*, Lawrence Erlbaum Associates, 1987, pp. 89–122.

[18] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research," *Advances in Psychology*, vol. 52, pp. 139–183, 1988, doi: 10.1016/S0166-4115(08)62386-9.

[19] J. Brooke, "SUS: A Quick and Dirty Usability Scale," *Usability Evaluation in Industry*, vol. 189, no. 194, pp. 4–7, 1996.

Research Methodology

Hypothesis

A comprehensive, interactive, visual platform improves exam performance on formal languages topics by 15-25% compared to existing tools, with strongest effects on relational understanding and conversion questions.

Study Protocol

Preparation

- Complete platform implementation
- Prepare example library and tutorials
- Get ethics approval and course instructor agreement
- Design exam questions covering: execution tracing, automaton design, conversions, theory/proofs

Baseline (Week 0)

- Recruit participants from course (N=80-100)
- Background survey: prior CS courses, programming experience, learning preferences
- Informed consent
- Stratified random assignment (matched on experience)

Intervention (Weeks 1-5)

Both Groups:

- Attend same lectures (instructor teaches both, no differential treatment)
- Same homework problem sets
- Same conceptual material

Control Group:

- Use existing tools: JFLAP for construction/simulation, online regex testers, paper exercises
- Access to all standard resources

Treatment Group:

- Unified platform for all homework and practice
- Tutorial session (Week 1, 30 min)
- Encouraged to explore examples and conversion visualizations

Data Collection:

- Weekly cognitive load survey (after homework, 5 min)
- Usage logs: time spent, features used, errors encountered (treatment only)
- Weekly quiz scores (both groups, track progress)

Exam Assessment (Week 6)

Exam Structure (designed with instructor):

- Part A: Execution (trace DFA/NFA/PDA on inputs)
- Part B: Design (construct automaton for specified language)
- Part C: Conversions (NFA $\longrightarrow$ DFA, regex $\longleftrightarrow$ automaton)
- Part D: Theory (proofs, closure properties, decidability)

Supplemental Assessment (after exam):

- Relational understanding: "Explain why every regex has equivalent DFA. Describe conversion process."
- Timed conversion: Standardized NFA $\longrightarrow$ minimal DFA (20 min)
- SUS questionnaire (treatment)

- Interviews: 15 per group (stratified by exam performance: 5 high, 5 mid, 5 low)

Analysis

Primary Analysis:

- t-test: exam total scores (control vs treatment)
- Effect size: Cohen's d
- Check assumptions: normality (Shapiro-Wilk), equal variance (Levene)

Secondary Analysis:

- ANCOVA: exam score group + prior_experience
- By question type: which parts show largest differences?
- Relational rubric scores: Mann-Whitney U test
- Conversion task: accuracy (chi-square), time (t-test)
- Cognitive load: NASA-TLX trajectory over weeks

Feature Analysis (treatment only):

- Regression: exam score conversion_viz_time + editor_time + simulation_replays + ...
- Identify high-value features

Qualitative:

- Interview transcription and thematic coding
- Compare themes between high/low performers
- Triangulate with quantitative findings

Validity Considerations

Internal: Random assignment, blinded grading (exams coded before scoring), same instructor/content

External: Single institution limits generalizability; replication recommended

Construct: Exam designed to test multiple competencies (not just memorization)

Ecological: Using real course exam (not artificial lab test) increases relevance

Original Approach

Exam-based evaluation: Most educational tools evaluated via separate tests. We use actual course exam for ecological validity and higher stakes.

Multi-factor platform: Not just addressing "fragmentation" but combining comprehensiveness, usability, visualization, interactivity in unified design.

Conversion emphasis: Explicit focus on algorithm visualization for learning relationships, not just simulation.

Short realistic study: 3-5 weeks feasible within semester constraints, not idealized full-semester intervention.

Assessment Details

Relational Understanding Rubric:

- **Level 0:** No answer or incorrect
- **Level 1:** Vague ("they're related")
- **Level 2:** Correct claim without justification ("DFA and regex are equivalent")
- **Level 3:** Procedural knowledge ("Thompson's construction converts regex to NFA")
- **Level 4:** Deep explanation with correctness proof and examples

Conversion Task:

- Standardized 5-state NFA (multiple transitions per state)

- Must produce minimal DFA
- Scored: correct structure (3 pts), correct transitions (3 pts), minimal (2 pts), clear notation (2 pts)

Interview Questions (selection):

- How did you approach learning automata theory?
- Describe a moment when concepts "clicked" for you
- How did you understand relationships between DFA/NFA/regex?
- (Treatment): Which platform features were most/least helpful?
- What would improve the learning experience?

Original Contribution

Contributions

C1: Comprehensive Educational Platform

Modern web-based tool spanning Chomsky hierarchy with focus on usability, visualization, and interactive learning (not just feature completeness).

C2: Exam-Based Evaluation

Using real course assessment for ecological validity. Most tool studies use artificial lab tests.

C3: Multi-Factor Design Principles

Platform combines: comprehensive coverage + modern UX + rich visualization + interactive construction + explicit relationships. Design principles for CS education tools.

C4: Open Resource

Freely available tool with examples and tutorials for adoption by other institutions.

Research Questions

RQ1: Exam Performance?

Hypothesis: 15-25% improvement in treatment group. Measured by course exam (instructor-designed, covers full curriculum). Strongest effects expected on conversion and relational understanding questions.

RQ2: Relational Understanding?

Measured by rubric-scored explanations. Expected: treatment group better articulates equivalences and conversion processes. Mechanism: explicit visualization of relationships + conversion algorithms.

RQ3: Feature Value?

Regression analysis of usage logs on performance. Predicted ranking: (1) conversion visualization, (2) interactive editor, (3) execution animation. Identifies high-impact features for design prioritization.

RQ4: Usability?

SUS > 75 indicates good usability. Qualitative feedback on learning experience. Assess adoption barriers and improvement opportunities.

Differentiation

vs. JFLAP Research:

Prior work measured satisfaction and adoption rates. We measure learning outcomes via exam performance. Modern web-based design vs desktop Java.

vs. Educational Tool Studies:

Most use separate pre/post tests. We use actual course exam (ecological validity). Short realistic intervention (3-5 weeks) vs idealized semester-long.

vs. Algorithm Visualization:

Domain-specific for formal languages with emphasis on conversions and relationships. Not generic algorithm viz.

Impact

Educational: Direct benefit to students, free tool for instructors, evidence-based design guidance

Research: Methodology for tool evaluation, design principles, publications (SIGCSE, ITiCSE, ACM TOCE)

Practical: Reduces barriers to learning formal languages, potential for wide adoption (web-based, open-source)